Scenario 1: Emergency Rescue Priority System

Code

```c
#include <stdio.h>

struct Rescue
{
    int id;
    char location[50];
    int people;
    int level;
    char resources[50];
    char team[50];
    char status[30];
};

int main()
{
    struct Rescue r[100], temp;
    int n;

    printf("Enter number of rescue requests: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        printf("\nRequest %d\n", i + 1);

        printf("Request ID: ");
        scanf("%d", &r[i].id);

        printf("Location: ");
        scanf(" %49[^\n]", r[i].location);

        printf("People affected: ");
        scanf("%d", &r[i].people);
```

```
        printf("Emergency Level (1-Low, 2-Medium, 3-Critical): ");
        scanf("%d", &r[i].level);

        printf("Required Resources: ");
        scanf(" %49[^\n]", r[i].resources);

        printf("Assigned Team: ");
        scanf(" %49[^\n]", r[i].team);

        printf("Status: ");
        scanf(" %29[^\n]", r[i].status);
    }


    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (r[j].level < r[j + 1].level ||
               (r[j].level == r[j + 1].level &&
                r[j].people < r[j + 1].people))
            {
                temp = r[j];
                r[j] = r[j + 1];
                r[j + 1] = temp;
            }
        }
    }

    printf("\n--- PRIORITIZED RESCUE REQUESTS ---\n");

    for (int i = 0; i < n; i++)
    {
        printf("\nRequest ID: %d\n", r[i].id);
        printf("Location: %s\n", r[i].location);
        printf("People Affected: %d\n", r[i].people);
        printf("Emergency Level: %d\n", r[i].level);
        printf("Resources: %s\n", r[i].resources);
        printf("Assigned Team: %s\n", r[i].team);
        printf("Status: %s\n", r[i].status);
```

```
    }

    return 0;
}
```

OUTPUT

```
Enter number of rescue requests: 3

Request 1
Request ID: AA1
Location: People affected: 6
Emergency Level (1-Low, 2-Medium, 3-Critical): 2
Required Resources: ambulance
Assigned Team: 2
Status: pending

Request 2
Request ID: AA4
Location: People affected: 5
Emergency Level (1-Low, 2-Medium, 3-Critical): 1
Required Resources: first aid kit
Assigned Team: 1
Status: waiting

Request 3
Request ID: AA2
Location: People affected: 15
Emergency Level (1-Low, 2-Medium, 3-Critical): 3
Required Resources: backup
Assigned Team: 4
Status: pending

--- PRIORITIZED RESCUE REQUESTS ---

Request ID: 0
Location: AA2
People Affected: 15
Emergency Level: 3
Resources: backup
Assigned Team: 4
Status: pending

Request ID: 0
Location: AA1
People Affected: 6
Emergency Level: 2
Resources: ambulance
Assigned Team: 2
Status: pending

Request ID: 0
Location: AA4
People Affected: 5
Emergency Level: 1
Resources: first aid kit
```

Scenario 2: Smart Campus Complaint System

CODE

```c
#include <stdio.h>

struct Complaint
{
    int id;
    char name[50];
    char category[30];
    char description[100];
    int priority;
    char status[30];
};

int main()
{
    struct Complaint c[100], temp;
    int n, pending = 0, resolved = 0, progress = 0;

    printf("Enter number of complaints: ");
    scanf("%d", &n);

    if (n < 1 || n > 100)
    {
        printf("Invalid number of complaints!\n");
        return 0;
    }

    for (int i = 0; i < n; i++)
    {
        printf("\nComplaint %d\n", i + 1);

        printf("Complaint ID: ");
        scanf("%d", &c[i].id);
```

```
        printf("Name: ");
        scanf(" %49[^\n]", c[i].name);

        printf("Category (Lab/Hostel/Network/etc.): ");
        scanf(" %29[^\n]", c[i].category);

        printf("Description: ");
        scanf(" %99[^\n]", c[i].description);

        printf("Priority (1-Low, 2-Medium, 3-High, 4-Critical): ");
        scanf("%d", &c[i].priority);

        printf("Status (Pending/InProgress/Resolved): ");
        scanf(" %29[^\n]", c[i].status);
    }

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (c[j].priority < c[j + 1].priority)
            {
                temp = c[j];
                c[j] = c[j + 1];
                c[j + 1] = temp;
            }
        }
    }

    printf("\n--- PRIORITIZED COMPLAINTS ---\n");

    for (int i = 0; i < n; i++)
    {
        printf("\nID: %d\n", c[i].id);
        printf("Name: %s\n", c[i].name);
        printf("Category: %s\n", c[i].category);
        printf("Description: %s\n", c[i].description);
        printf("Priority: %d\n", c[i].priority);
        printf("Status: %s\n", c[i].status);
```

```
        // Count complaints by status
        if (c[i].status[0] == 'P' || c[i].status[0] == 'p')
            pending++;
        else if (c[i].status[0] == 'R' || c[i].status[0] == 'r')
            resolved++;
        else if (c[i].status[0] == 'I' || c[i].status[0] == 'i')
            progress++;
    }

    printf("\n--- COMPLAINT SUMMARY ---\n");
    printf("Total complaints: %d\n", n);
    printf("Pending: %d\n", pending);
    printf("In Progress: %d\n", progress);
    printf("Resolved: %d\n", resolved);

    return 0;
}
```

OUTPUT

```
Complaint 1
Complaint ID: AA121
Name: Category (Lab/Hostel/Network/etc.): LAB
Description: COMPUTRER NOT WORKING
Priority (1-Low, 2-Medium, 3-High, 4-Critical): 2
Status (Pending/InProgress/Resolved): INPROGRESS

Complaint 2
Complaint ID: AA231
Name: Category (Lab/Hostel/Network/etc.): HOSTEL
Description: FAN NOT WORKING
Priority (1-Low, 2-Medium, 3-High, 4-Critical): 3
Status (Pending/InProgress/Resolved): PENDING

Complaint 3
Complaint ID: AA222
Name: Category (Lab/Hostel/Network/etc.): LAB
Description: CPU NOT WORKING
Priority (1-Low, 2-Medium, 3-High, 4-Critical): 4
Status (Pending/InProgress/Resolved): PENDING

--- PRIORITIZED COMPLAINTS ---

ID: 0
Name: AA222
Category: LAB
Description: CPU NOT WORKING
Priority: 4
Status: PENDING

ID: 0
Name: AA231
Category: HOSTEL
Description: FAN NOT WORKING
Priority: 3
Status: PENDING

ID: 0
Name: AA121
Category: LAB
Description: COMPUTRER NOT WORKING
Priority: 2
Status: INPROGRESS

--- COMPLAINT SUMMARY ---
Total complaints: 3
```

Scenario 3: Smart Hospital Emergency Queue System

Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100
#define DOCTORS 5
#define BEDS 10

typedef struct {
    char id[20];
    char name[50];
    int age;
    char condition[100];
    int priority;
    char status[20];
    int doctor;
    int bed;
} Patient;

Patient patients[MAX];
int count = 0;

const char *doctorNames[DOCTORS] = {
    "Dr. Kumar", "Dr. Priya", "Dr. Ravi",
    "Dr. Meena", "Dr. Arun"
};

int doctorBusy[DOCTORS] = {0};
int bedBusy[BEDS] = {0};

void saveData(void) {
```

```
    FILE *fp = fopen("patients.dat", "wb");
    if (fp == NULL) {
        printf("Error saving patient records!\n");
        return;
    }

    fwrite(&count, sizeof(int), 1, fp);
    fwrite(patients, sizeof(Patient), count, fp);
    fclose(fp);
}

void loadData(void) {
    FILE *fp = fopen("patients.dat", "rb");
    if (fp == NULL)
        return;

    if (fread(&count, sizeof(int), 1, fp) != 1 ||
        count < 0 || count > MAX) {
        count = 0;
        fclose(fp);
        return;
    }

    if (fread(patients, sizeof(Patient), count, fp)
        != (size_t)count) {
        count = 0;
        fclose(fp);
        return;
    }

    fclose(fp);

    for (int i = 0; i < count; i++) {
        if (strcmp(patients[i].status, "Treatment") == 0) {
            if (patients[i].doctor >= 0 &&
                patients[i].doctor < DOCTORS)
                doctorBusy[patients[i].doctor] = 1;

            if (patients[i].bed >= 0 &&
                patients[i].bed < BEDS)
```

```
                bedBusy[patients[i].bed] = 1;
        }
    }
}

void readLine(const char *message, char *buffer, size_t size) {
    printf("%s", message);
    if (fgets(buffer, (int)size, stdin) != NULL) {
        buffer[strcspn(buffer, "\n")] = '\0';
    } else {
        buffer[0] = '\0';
    }
}

int readInt(const char *message) {
    char buffer[100];
    char *end;
    long value;

    while (1) {
        readLine(message, buffer, sizeof(buffer));
        value = strtol(buffer, &end, 10);

        while (isspace((unsigned char)*end))
            end++;

        if (buffer[0] != '\0' && *end == '\0' &&
            value >= -2147483647L && value <= 2147483647L)
            return (int)value;

        printf("Enter a valid number.\n");
    }
}

/* Demo keyword-based priority assignment */
int assignPriority(const char *condition) {
    char text[100];
    size_t i;

    snprintf(text, sizeof(text), "%s", condition);
```

```
    for (i = 0; text[i] != '\0'; i++)
        text[i] = (char)tolower((unsigned char)text[i]);

    if (strstr(text, "cardiac arrest") ||
        strstr(text, "heart attack") ||
        strstr(text, "severe accident") ||
        strstr(text, "unconscious") ||
        strstr(text, "not breathing") ||
        strstr(text, "severe bleeding"))
        return 1;

    if (strstr(text, "breathing difficulty") ||
        strstr(text, "breathlessness") ||
        strstr(text, "serious injury") ||
        strstr(text, "fracture") ||
        strstr(text, "heavy bleeding"))
        return 2;

    if (strstr(text, "fever") ||
        strstr(text, "vomiting") ||
        strstr(text, "moderate injury") ||
        strstr(text, "abdominal pain"))
        return 3;

    return 4;
}

const char *priorityName(int priority) {
    switch (priority) {
        case 1: return "Critical";
        case 2: return "High";
        case 3: return "Medium";
        default: return "Low";
    }
}

int findPatient(const char *id) {
    for (int i = 0; i < count; i++) {
        if (strcmp(patients[i].id, id) == 0)
```

```
            return i;
    }
    return -1;
}

void registerPatient(void) {
    if (count >= MAX) {
        printf("Patient storage is full!\n");
        return;
    }

    Patient p;
    memset(&p, 0, sizeof(p));

    int nextId = 101;
    char generatedId[20];

    do {
        snprintf(generatedId, sizeof(generatedId), "P%d", nextId++);
    } while (findPatient(generatedId) != -1);

    strcpy(p.id, generatedId);

    printf("\n--- Register Patient ---\n");
    printf("Generated Patient ID: %s\n", p.id);

    readLine("Enter patient name: ", p.name, sizeof(p.name));

    do {
        p.age = readInt("Enter age: ");
        if (p.age < 0 || p.age > 120)
            printf("Enter an age from 0 to 120.\n");
    } while (p.age < 0 || p.age > 120);

    readLine("Enter condition: ", p.condition,
             sizeof(p.condition));

    p.priority = assignPriority(p.condition);
    strcpy(p.status, "Waiting");
    p.doctor = -1;
```

```
    p.bed = -1;

    patients[count++] = p;
    saveData();

    printf("\nPatient registered successfully!\n");
    printf("Patient ID : %s\n", p.id);
    printf("Priority   : %d - %s\n",
           p.priority, priorityName(p.priority));
    printf("Status     : Waiting\n");

    printf("\nNote: Priority is estimated from keywords.\n");
    printf("Medical staff must verify urgency in real use.\n");
}

void displayPatient(int i) {
    printf("\nID       : %s\n", patients[i].id);
    printf("Name     : %s\n", patients[i].name);
    printf("Age      : %d\n", patients[i].age);
    printf("Condition: %s\n", patients[i].condition);
    printf("Priority : %d - %s\n",
           patients[i].priority,
           priorityName(patients[i].priority));
    printf("Status   : %s\n", patients[i].status);

    if (patients[i].doctor >= 0 &&
        patients[i].doctor < DOCTORS)
        printf("Doctor   : %s\n",
               doctorNames[patients[i].doctor]);

    if (patients[i].bed >= 0 &&
        patients[i].bed < BEDS)
        printf("Bed      : B%d\n", patients[i].bed + 1);
}

void displayQueue(void) {
    int order[MAX];

    if (count == 0) {
        printf("\nNo patient records available.\n");
```

```
        return;
    }

    for (int i = 0; i < count; i++)
        order[i] = i;

    /* Sort by priority; earlier registrations come first
       when priorities are equal. */
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (patients[order[j]].priority >
                patients[order[j + 1]].priority) {
                int temp = order[j];
                order[j] = order[j + 1];
                order[j + 1] = temp;
            }
        }
    }

    printf("\n========== PATIENT QUEUE ==========\n");
    printf("%-8s %-18s %-8s %-12s %-12s\n",
           "ID", "Name", "Priority", "Level", "Status");

    for (int i = 0; i < count; i++) {
        int k = order[i];
        printf("%-8s %-18s %-8d %-12s %-12s\n",
               patients[k].id, patients[k].name,
               patients[k].priority,
               priorityName(patients[k].priority),
               patients[k].status);
    }
}

int freeDoctor(void) {
    for (int i = 0; i < DOCTORS; i++)
        if (!doctorBusy[i])
            return i;
    return -1;
}
```

```c
int freeBed(void) {
    for (int i = 0; i < BEDS; i++)
        if (!bedBusy[i])
            return i;
    return -1;
}

void callNextPatient(void) {
    int selected = -1;

    for (int p = 1; p <= 4 && selected == -1; p++) {
        for (int i = 0; i < count; i++) {
            if (patients[i].priority == p &&
                strcmp(patients[i].status, "Waiting") == 0) {
                selected = i;
                break;
            }
        }
    }

    if (selected == -1) {
        printf("\nNo waiting patients.\n");
        return;
    }

    int d = freeDoctor();
    int b = freeBed();

    if (d == -1 || b == -1) {
        printf("\nNo available doctor or bed.\n");
        printf("Patient %s remains in the waiting queue.\n",
               patients[selected].id);
        return;
    }

    patients[selected].doctor = d;
    patients[selected].bed = b;
    doctorBusy[d] = 1;
    bedBusy[b] = 1;
    strcpy(patients[selected].status, "Treatment");
```

```
    printf("\nNext patient called successfully!\n");
    displayPatient(selected);
    saveData();
}

void updateStatus(void) {
    char id[20];
    readLine("Enter patient ID: ", id, sizeof(id));

    int i = findPatient(id);

    if (i == -1) {
        printf("Patient not found.\n");
        return;
    }

    printf("1. Mark as Treatment\n");
    printf("2. Mark as Completed\n");
    int choice = readInt("Choose status: ");

    if (choice == 1) {
        if (strcmp(patients[i].status, "Completed") == 0) {
            printf("Completed patient cannot be reopened here.\n");
            return;
        }

        if (strcmp(patients[i].status, "Treatment") == 0) {
            printf("Patient is already in treatment.\n");
            return;
        }

        int d = freeDoctor();
        int b = freeBed();

        if (d == -1 || b == -1) {
            printf("No available doctor or bed.\n");
            return;
        }
```

```
        patients[i].doctor = d;
        patients[i].bed = b;
        doctorBusy[d] = 1;
        bedBusy[b] = 1;
        strcpy(patients[i].status, "Treatment");
        printf("Patient moved to treatment.\n");

    } else if (choice == 2) {
        if (strcmp(patients[i].status, "Waiting") == 0) {
            printf("A waiting patient cannot be completed.\n");
            return;
        }

        if (patients[i].doctor >= 0 &&
            patients[i].doctor < DOCTORS)
            doctorBusy[patients[i].doctor] = 0;

        if (patients[i].bed >= 0 &&
            patients[i].bed < BEDS)
            bedBusy[patients[i].bed] = 0;

        patients[i].doctor = -1;
        patients[i].bed = -1;
        strcpy(patients[i].status, "Completed");
        printf("Treatment completed. Resources released.\n");

    } else {
        printf("Invalid choice.\n");
        return;
    }

    saveData();
}

void searchPatient(void) {
    char id[20];
    readLine("Enter patient ID: ", id, sizeof(id));

    int i = findPatient(id);
```

```
    if (i == -1)
        printf("Patient not found.\n");
    else
        displayPatient(i);
}

void showSummary(void) {
    int waiting = 0, treatment = 0, completed = 0;
    int levels[5] = {0};

    for (int i = 0; i < count; i++) {
        if (strcmp(patients[i].status, "Waiting") == 0)
            waiting++;
        else if (strcmp(patients[i].status, "Treatment") == 0)
            treatment++;
        else if (strcmp(patients[i].status, "Completed") == 0)
            completed++;

        levels[patients[i].priority]++;
    }

    printf("\n======= EMERGENCY SUMMARY =======\n");
    printf("Total registered : %d\n", count);
    printf("Waiting          : %d\n", waiting);
    printf("In treatment     : %d\n", treatment);
    printf("Completed        : %d\n", completed);

    printf("\nPriority-wise totals:\n");
    for (int p = 1; p <= 4; p++)
        printf("%-10s : %d\n", priorityName(p), levels[p]);

    printf("\nAvailable doctors: ");
    int available = 0;
    for (int d = 0; d < DOCTORS; d++) {
        if (!doctorBusy[d]) {
            printf("%s  ", doctorNames[d]);
            available++;
        }
    }
    printf("\nCount: %d of %d\n", available, DOCTORS);
```

```
    printf("Available beds  : ");
    available = 0;
    for (int b = 0; b < BEDS; b++) {
        if (!bedBusy[b]) {
            printf("B%d ", b + 1);
            available++;
        }
    }
    printf("\nCount: %d of %d\n", available, BEDS);
}

int main(void) {
    loadData();

    int choice;

    printf("========================================\n");
    printf("   SMART HOSPITAL EMERGENCY QUEUE\n");
    printf("========================================\n");

    while (1) {
        printf("\n1. Register new patient\n");
        printf("2. Display priority queue\n");
        printf("3. Call next highest-priority patient\n");
        printf("4. Update patient status\n");
        printf("5. Search patient\n");
        printf("6. Emergency department summary\n");
        printf("7. Exit\n");

        choice = readInt("Enter your choice: ");

        switch (choice) {
            case 1: registerPatient(); break;
            case 2: displayQueue(); break;
            case 3: callNextPatient(); break;
            case 4: updateStatus(); break;
            case 5: searchPatient(); break;
            case 6: showSummary(); break;
            case 7:
```

```
                saveData();
                printf("Records saved. Exiting program.\n");
                return 0;
            default:
                printf("Invalid choice. Enter 1 to 7.\n");
        }
    }
}
```

Output

```
======================================
   SMART HOSPITAL EMERGENCY QUEUE
======================================

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 1

--- Register Patient ---
Generated Patient ID: P101
Enter patient name: akarshana
Enter age: 18
Enter condition: cold

Patient registered successfully!
Patient ID : P101
Priority   : 4 - Low
Status     : Waiting

Note: Priority is estimated from keywords.
Medical staff must verify urgency in real use.

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 2

---------- PATIENT QUEUE ----------
ID       Name               Priority Level        Status
P101     akarshana          4        Low          Waiting

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 3
```

```
Next patient called successfully!

ID       : P101
Name     : akarshana
Age      : 18
Condition: cold
Priority : 4 - Low
Status   : Treatment
Doctor   : Dr. Kumar
Bed      : B1

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 4
Enter patient ID: p101
Patient not found.

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 5
Enter patient ID: P101

ID       : P101
Name     : akarshana
Age      : 18
Condition: cold
Priority : 4 - Low
Status   : Treatment
Doctor   : Dr. Kumar
Bed      : B1

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
```

```
Age       : 18
Condition: cold
Priority : 4 - Low
Status    : Treatment
Doctor    : Dr. Kumar
Bed       : B1

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 6

======= EMERGENCY SUMMARY =======
Total registered : 1
Waiting          : 0
In treatment     : 1
Completed        : 0

Priority-wise totals:
Critical   : 0
High       : 0
Medium     : 0
Low        : 1

Available doctors: Dr. Priya  Dr. Ravi  Dr. Meena  Dr. Arun
Count: 4 of 5
Available beds  : B2 B3 B4 B5 B6 B7 B8 B9 B10
Count: 9 of 10

1. Register new patient
2. Display priority queue
3. Call next highest-priority patient
4. Update patient status
5. Search patient
6. Emergency department summary
7. Exit
Enter your choice: 7
Records saved. Exiting program.
PS C:\Users\NETRA MS\OneDrive\Desktop\c codecraft 26>
```